NCDawareRank: a Novel Ranking Method that Exploits the Decomposable Structure of the Web

Athanasios N. Nikolakopoulos
nikolako@ceid.upatras.gr

John D. Garofalakis
garofala@ceid.upatras.gr

Computer Engineering and Informatics Department, University of Patras
Computer Technology Institute and Press "Diophantus"
GR-26504 Patras, Greece

ABSTRACT

Research about the topological characteristics of the hyperlink graph has shown that Web possesses a nested block structure, indicative of its innate hierarchical organization. This crucial observation opens the way for new approaches that can usefully regard Web as a *Nearly Completely Decomposable*(NCD) system; In recent years, such approaches gave birth to various efficient methods and algorithms that exploit NCD from a computational point of view and manage to considerably accelerate the extraction of the *PageRank* vector. However, very little have been done towards the qualitative exploitation of NCD.

In this paper we propose *NCDawareRank*, a novel ranking method that uses the intuition behind NCD to generalize and refine PageRank. NCDawareRank considers both the link structure and the hierarchical nature of the Web in a way that preserves the mathematically attractive characteristics of PageRank and at the same time manages to successfully resolve many of its known problems, including Web Spamming Susceptibility and Biased Ranking of Newly Emerging Pages. Experimental results show that NCDawareRank is more resistant to direct manipulation, alleviates the problems caused by the sparseness of the link graph and assigns more reasonable ranking scores to newly added pages, while maintaining the ability to be easily implemented on a large-scale and in a computationally efficient manner.

Categories and Subject Descriptors

H.3.3 [**Information Search and Retrieval**]: Search process; G.1.3 [**Numerical Linear Algebra**]: Sparse Structured and Very Large Systems; G.2.2 [**Probability and Statistics**]: Markov processes

Keywords

Link Analysis; Ranking; Link Spamming; Sparsity; Near Complete Decomposability; NCDawareRank; PageRank

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
WSDM'13, February 4–8, 2013, Rome, Italy.
Copyright 2013 ACM 978-1-4503-1869-3/13/02 ...$15.00.

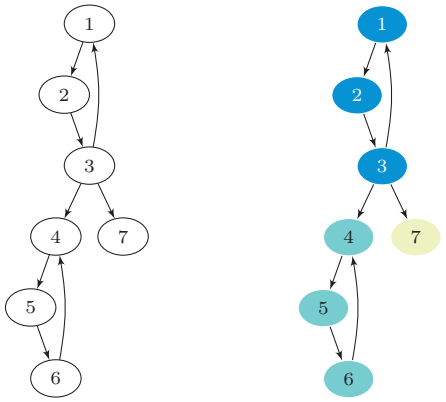
1. INTRODUCTION

For years now, Web searching has become an integral part of everyone's daily activities. Due to the astonishing amount of information available on the Web and because of the fact that unlike traditional text collections, Web is dynamic and its content exhibits highly variable quality, classic information retrieval methods can only be used by search engines to a small extend. This fact generates the need for an absolute measure of importance for Web pages, that can be used to improve the performance of Web search.

Link Analysis algorithms such as the celebrated *PageRank*, try to answer this need by using the link structure of the Web to assign authoritative weights to Web pages. PageRank's approach is based on the assumption that links convey human endorsement. For example, the existence of a link from page 3 to page 4 in Figure 1(a) is viewed as a testimonial of the importance of page 4. In their original paper, Page et al.[25] imagined of a *random surfer* who with probability α follows the links of a Web page uniformly at random, and with probability $1-\alpha$ jumps to a different page according to a given distribution. Then, following this intuitive metaphor, the overall importance of a page will be equal to the fraction of time this random surfer spends on it, in the long run.

Although PageRank has proven to be a very effective paradigm for ranking Web pages and it is believed to be part of the ranking functions of most today's commercial search engines, it suffers from a number of problems. One of these problems is susceptibility to manipulation through link spamming; a form of Web spamming that involves the artificial creation of a large number of pages that carefully funnel all their rank towards a single page of interest [13, 24]. Spam Web pages represent a significant fraction of the overall pages in most domains (in some, this fraction is alarming; according to [24], about 70% of .biz domain can be characterized as spam) and if we take into account the fact that Web search is the *de facto* control point for e-commerce, the economic incentive behind high ranking immediately justifies the need for spam resistant ranking schemes.

Another problem of PageRank comes from the low density of the link graph. It is well known that the link distribution of the Web graph generally follows a power law [14]. This leads to a sparse hyperlink matrix. Furthermore, in [26] it was observed that such link distributions caused the probability values produced by PageRank to decay according to a power law, making most of the pages unable to obtain reasonable score. The latter is especially true for the newly



(a) "flat" web (b) NCD web

Figure 1: Tiny web graph

added pages which usually have too-few incoming links, and thus cannot receive reasonable ranking [9].

We believe that one of the main causes of the previously mentioned problems is that PageRank, as in fact most link analysis algorithms, approach Web in a "flat" way. Research about the topological structure of the Web has shown that the hyperlink graph has a nested block structure [19], with domains, hosts and websites, introducing intermediate levels of affiliation and revealing an interesting macro-structure [3, 12]. Thus, the Web just like many naturally emerging complex systems has a hierarchical nature. According to Simon [30] this is no accident; basically all viable complex systems, be they physical, social, biological, or artificial, share the property of having a *nearly completely decomposable* architecture: they are organized into hierarchical layers of blocks, sub-blocks, sub-sub-blocks and so on, in such a way that interactions among elements belonging to the same block are much stronger than interactions between elements belonging to different blocks.

The analysis of decomposable systems has been pioneered by Simon and Ando [29], who reported on state aggregation in linear models of economic systems, but the versatility of Simon's idea has permitted the theory to be used with noticeable success in many complex problems originated from evolutionary biology, social sciences, cognitive science, management, etc. The introduction of near complete decomposability (NCD) in the fields of computer science and engineering is due to Courtois [11] who achieved the full mathematical development of the theory, and applied it with great originality in a number of queueing and computer system performance problems.

In recent years, the decomposability of the Web has been exploited mainly from a computational point of view. The "mathematical fingerprint" of NCD in the PageRank problem is the special spectral structure [11, 23] of the stochastic matrix corresponding to the random walk on the Web graph. This property opens the way for the use of numerical methods like Iterative Aggregation/Disaggregation (IAD) [27] that can be used to accelerate the computation of the PageRank vector considerably. Many researchers have followed such approaches with promising results. Kamvar et al.[19] use the first two steps of the general IAD method to obtain an initial vector for subsequent Power Method iterations. Zhu et al.[33] propose a distributed PageRank computation algorithm based on IAD methods. Langville and Meyer [21] use a two-block IAD to accelerate the updating of PageRank vector, and Ipsen et al.[17] analyse its asymptotic convergence. Recently, Cevahir et al.[8] used site based partitioning techniques in order to accelerate PageRank.

However, little have been done to exploit decomposability from a qualitative point of view. Knowing that a complex system possesses the property of NCD, points the way towards a more appropriate modelling approach and a mathematical analysis, which highlights the system's endemic characteristics, gives us invaluable insight to its behaviour and consequently, provides us a theoretical framework to develop algorithms and methods that materialize this insight from a qualitative, computational as well as conceptual angle.

The main question we try to address in this work is *"How could someone incorporate the concept of NCD to the basic PageRank model in a way that refines and generalizes it while preserving its efficiency?"*. For example, if we have the information that the tiny Web graph of Figure 1(a) can be decomposed like in Figure 1(b)[1], in what way can we utilize this to achieve better ranking, without obscuring the simplicity and the clarity of PageRank's approach?

Summary of Contributions.

The main contribution of this paper is the proposal of *NCDawareRank*, a novel ranking measure which:

- provides a theoretical framework that enables the exploitation of Web's innately decomposable structure in a computationally efficient way.
- can serve as a generalization of PageRank that enhances its expressiveness while inheriting its attractive mathematical characteristics and approach.
- displays low sensitivity to the problems caused by the sparsity of the Web graph and treats newly added pages more fairly.
- exhibits resistance to direct manipulation through link spamming.

The rest of the paper is organized as follows. In Section 2, we outline the basic idea behind NCDawareRank and we briefly discuss the nature of the basic NCD blocks. In Section 3, we develop the mathematical framework of NCDawareRank. Our experimental results are presented in Section 4. Finally, Section 5 concludes this paper and outlines directions for future work.

2. EXPLOITING WEB'S DECOMPOSABILITY: THE INTUITION

2.1 From PageRank to NCDawareRank

Underlying the definition of PageRank is the assumption that the existence of a link from page u to page v testifies the importance of page v. Furthermore, the amount of importance conferred to page v is proportional to the importance of page u and inversely proportional to the number of pages u links to.

[1]same coloured nodes represent pages belonging to the same block

To formulate PageRank's basic idea with a mathematical model, we view the Web as a directed graph with Web pages as vertices and hyperlinks as edges. Given this graph, we can construct a *row-normalized hyperlink matrix* $\mathbf{H}$, whose element H_{uv} is one over the outdegree of u if there is a link from u to v, or zero otherwise.

When the graph contains *dangling nodes* (i.e nodes with no outgoing links), the corresponding rows of $\mathbf{H}$ will be entirely made of zeros, which makes $\mathbf{H}$ a substochastic matrix. A simple adjustment that fixes this problem, originally proposed by Page et al.[25], is to replace the $\mathbf{0}^\intercal$ rows with the uniform distribution over all pages, thereby transforming $\mathbf{H}$ to a stochastic matrix.

A second adjustment is needed to verify that the final matrix is irreducible and aperiodic, so that it possesses a unique positive stationary probability distribution. That is ensured by the introduction of the *damping factor* α and a *teleportation* matrix $\mathbf{E}$, usually defined by $\mathbf{E} = \mathbf{e}\mathbf{v}^\intercal$, where $\mathbf{v}$ a probability vector with non-negative elements. The resulting matrix is given by:

$$\mathbf{G} = \alpha\mathbf{H} + (1-\alpha)\mathbf{E} \tag{1}$$

PageRank vector is the unique stationary distribution of the Markov chain corresponding to matrix $\mathbf{G}$.

The choice of the damping factor α is very important and has recently received great research attention [1, 7, 22]. This parameter determines how often the Markov chain follows the hyperlink graph rather than jumping at a random page. Picking a very small damping factor ignores the link structure of the Web and thus results in uninformative ranking vectors. On the other hand, as α gets closer to 1, the Web component becomes increasingly important.

While this may seem intuitively preferable, picking an α very close to 1 results in a number of problems: From a computational point of view, the number of iterations till convergence required by the Power Method and its variants (commonly used for the extraction of the PageRank vector) grows with α and the computation of the ranking vector becomes numerically ill-conditioned [18]. Furthermore, from a qualitative prospective, various studies also indicate that damping factors close to 1 result into counterintuitive ranking vectors where the Web's core component is assigned null rank and all the PageRank gets concentrated mostly in irrelevant nodes [1, 6, 7].

Several ranking methods proposed lately try to address the damping factor issue. Boldi in [4] proposed the *TotalRank* algorithm which aims at eliminating PageRank's dependency on the arbitrarily chosen parameter α by integrating the ranking vector over the entire range of possible damping factors. Baeza-Yates et al.[2] introduce a family of link based ranking algorithms parametrised by the selection of a *damping function* that describes how rapidly the importance of paths decays as the path length increases. *HyperRank* assumes general hyperbolic damping, whereas *LinearRank* assumes a linear damping function. In a recent work, Constantine and Gleich [10, 15] generalize the standard random surfer model and propose a ranking method that considers the influence of a population of random surfers. Each surfer chooses its own teleportation parameter from a probability distribution and thus the ranking vector produced is random.

Exploiting Decomposability. Our goal is to introduce an approach that tries to address this fundamental issue by exploiting the fact that Web can be viewed as a nearly completely decomposable system. The partitioning of the Web into NCD blocks introduces new levels of abstraction, which we aim to exploit without creating coarser-level graph models that may obscure the direct link structure of the Web and hide valuable information. In the core of NCDawareRank is the idea that the existence of a link suggests *multiple levels of proximity* corresponding to the different decompositions of the underlying space. This basic idea can be used to attack the importance propagation issue from a different angle.

As explained above, quality and computational reasons force PageRank to keep the parameter α not so close to 1. This, however, leaves $1-\alpha$ of the rank spread uniformly on every page of the Web, in a hardly meaningful way. The basic idea behind our method is *a fraction of the importance of a page that is scattered throughout the Web, to be propagated instead, to the NCD blocks that contain the pages it links to.* In other words, the existence of an outgoing link, except for justifying the importance of the page it points to, "hints" about the importance of the block that contains this page. But before we explain the details behind the use of Decomposability in generalizing and refining PageRank, we must first describe the nature of the basic NCD blocks.

2.2 The basic NCD blocks

PageRank's view, while considering the direct structure of the Web, disregards the fact that Web pages are organized into Web sites, and these sites many times prove to be the real unit of interest. Bharat et al.[3] present a number of convincing arguments that make websites the appropriate level of abstraction to study Web's macro-structure. When compared to an individual page, a website can many times provide a wealth of semantic information. Pages included in a website are usually authored by the same person or group of persons and their content is often highly correlated. This fact is directly reflected to the topological properties of the Web graph; websites contain higher density of hyperlinks inside them (about 75% according to the authors in [16]) and lower density of edges in between. This is why websites have usually been used as the standard unit of partition in most NCD-based approaches to accelerate PageRank. For the same reasons, we choose to use websites as the basic blocks of the nearly completely decomposable Web.

3. NCDAWARERANK

3.1 Our Model

In order to model mathematically the basic idea of NCDawareRank, we introduce a new parameter μ and an associated *Inter-Level Proximity Matrix* $\mathbf{M}$. Before we proceed further, we need to define the parameters of our problem.

- Let $\mathcal{W}$ be the universe of Web pages and denote $n = |\mathcal{W}|$.
- Consider a Web page u in $\mathcal{W}$. We denote $\mathcal{G}_u$ to be the set of pages that can be visited in a single step from u. Clearly, $d_u \triangleq |\mathcal{G}_u|$ is the out-degree of u, i.e. the number of outgoing links of u.
- We consider a partition $\{\mathcal{A}_1, \mathcal{A}_2, \ldots, \mathcal{A}_N\}$ of $\mathcal{W}$. Each set of pages $\mathcal{A}_I$ is referred to as an *NCD block*. Obviously, it holds that for every Web page u there exists

a unique K such that $u \in \mathcal{A}_K$. We also use $\mathcal{A}_{(u)}$ to denote the unique block that includes page u.

- We define $\mathcal{X}_u$ to be the set of the *proximal* pages of u, i.e the union of the NCD blocks that contain u and the pages it links to. Formally, the set $\mathcal{X}_u$ is defined by:

$$\mathcal{X}_u \triangleq \bigcup_{w \in (u \cup \mathcal{G}_u)} \mathcal{A}_{(w)}$$

- Finally, we use N_u to denote the number of different blocks in $\mathcal{X}_u$.

The hyperlink matrix $\mathbf{H}$, as in the PageRank Model, is a row normalized version of the adjacency matrix induced by the Web graph, and its uv^{th} element is defined as follows:

$$H_{uv} \triangleq \begin{cases} \frac{1}{d_u} & \text{if } v \in \mathcal{G}_u \\ 0 & \text{otherwise} \end{cases}$$

Matrix $\mathbf{H}$ is assumed to be a row-stochastic matrix. The matter of dangling nodes is considered fixed through some sort of stochasticity adjustment.

The uv^{th} element of the Inter-Level Proximity matrix $\mathbf{M}$ is defined as follows:

$$M_{uv} \triangleq \begin{cases} \frac{1}{N_u |\mathcal{A}_{(v)}|} & \text{if } v \in \mathcal{X}_u \\ 0 & \text{otherwise} \end{cases}$$

For example, the matrix $\mathbf{M}$ that corresponds to the tiny Web graph of Figure 1(b) where we see 3 NCD blocks is:

$$\mathbf{M} = \begin{bmatrix} 1/3 & 1/3 & 1/3 & 0 & 0 & 0 & 0 \\ 1/3 & 1/3 & 1/3 & 0 & 0 & 0 & 0 \\ 1/9 & 1/9 & 1/9 & 1/9 & 1/9 & 1/9 & 1/3 \\ 0 & 0 & 0 & 1/3 & 1/3 & 1/3 & 0 \\ 0 & 0 & 0 & 1/3 & 1/3 & 1/3 & 0 \\ 0 & 0 & 0 & 1/3 & 1/3 & 1/3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

NCDawareRank also uses a teleportation matrix $\mathbf{E}$, as a remedy to ensure that the underlying Markov chain, corresponding to the final matrix, is irreducible and aperiodic and thus has a unique positive stationary probability distribution.

One very simple and convenient way to define $\mathbf{E}$ is the following:

$$\mathbf{E} \triangleq \mathbf{e}\mathbf{v}^{\intercal}$$

where vector $\mathbf{v}^{\intercal}$ is a probability vector that first distributes the rank evenly between the ncd blocks and then in each block evenly to the included pages. Of course, as in PageRank, one can construct a *personalized* version of the teleportation matrix, using a different stochastic vector instead of $\mathbf{v}^{\intercal}$.

Finally, the resulting matrix which we denote $\mathbf{P}$ is expressed by:

$$\mathbf{P} = \eta\mathbf{H} + \mu\mathbf{M} + (1 - \eta - \mu)\mathbf{E} \tag{2}$$

Parameter η controls the fraction of importance delivered to the outgoing links and parameter μ controls the fraction of importance that will be propagated to the proximal pages. In order to ensure the irreducibility and aperiodicity of the final stochastic matrix, $\eta + \mu$ must be less than 1. This leaves $1 - \eta - \mu$ of importance scattered throughout the Web through matrix $\mathbf{E}$.

Remark 1. Notice that the standard PageRank model can be seen as a special case of our framework in two different ways: When there is only one block containing all the nodes, matrix $\mathbf{M}$ collapses into the standard teleportation matrix, and NCDawareRank becomes PageRank. The latter is also true if we have n ncd blocks, each of which contains a single node; in this case, the inter-level proximity matrix coincides with the hyperlink matrix $\mathbf{H}$.

3.2 The Algorithm: Theory and Implementation Considerations

In a world where Web counts several billions of pages, every ranking method that realistically wants to attack the problem, needs to ensure its large scale implementability.

In our case it is crucial to be able to express matrix $\mathbf{P}$ as the sum of sparse matrices in order to efficiently compute its steady state distribution. The basic difference between PageRank and NCDawareRank from a computational point of view is the introduction of matrix $\mathbf{M}$. If we want our ranking system to be applicable in realistic scenarios, we need to ensure that matrix $\mathbf{M}$ can be handled efficiently.

3.2.1 *The Inter-Level Proximity Matrix* M

Matrix $\mathbf{M}$ is by definition a sparse matrix. For realistic purposes though, it is not sparse enough! Fortunately, $\mathbf{M}$ has a very special structure that can be exploited in order to achieve efficient storage and computability as well as other advantages. The key observation is that matrix $\mathbf{M}$ can be expressed as a product of 2 extremely sparse matrices, $\mathbf{R}$ and $\mathbf{A}$, defined below:

Matrix $\mathbf{A} \in \mathbb{R}^{N \times n}$ is defined as follows:

$$\mathbf{A} \triangleq \begin{bmatrix} \mathbf{e}^{\intercal}_{|\mathcal{A}_1|} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \mathbf{e}^{\intercal}_{|\mathcal{A}_2|} & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{e}^{\intercal}_{|\mathcal{A}_3|} & \cdots & \mathbf{0} \\ \vdots & \vdots & \vdots & \ddots & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \cdots & \mathbf{e}^{\intercal}_{|\mathcal{A}_N|} \end{bmatrix}$$

where $\mathbf{e}^{\intercal}_{|\mathcal{A}_k|}$ denotes a row vector in $\mathbb{R}^{|\mathcal{A}_k|}$ whose elements are all 1. In order to define the other factor of matrix $\mathbf{M}$ we first consider a vector

$$\mathbf{c} = \begin{pmatrix} 1/|\mathcal{A}_1| & 1/|\mathcal{A}_2| & \cdots & 1/|\mathcal{A}_N| \end{pmatrix}^{\intercal}$$

and a row normalized matrix $\mathbf{R}' \in \mathbb{R}^{n \times N}$ whose rows correspond to pages and columns to blocks and its ij^{th} element is equal to $\frac{1}{N_u}$ if $\mathcal{A}_j \in \mathcal{X}_u$, zero otherwise.

Now, if $\mathbf{R}'_{*\mathbf{j}}$, denotes the jth column of matrix $\mathbf{R}'$, and c_i the ith element of $\mathbf{c}$, we can define the matrix $\mathbf{R}$ as follows:

$$\mathbf{R} \triangleq \begin{bmatrix} c_1\mathbf{R}'_{*\mathbf{1}} & c_2\mathbf{R}'_{*\mathbf{2}} & \cdots & c_N\mathbf{R}'_{*\mathbf{N}} \end{bmatrix}$$

From the definition of these matrices it is clear that it holds $\mathbf{M} = \mathbf{R} \times \mathbf{A}$. For the sake of example, these matrices for the tiny Web graph of Figure 1(b) are:

$$\mathbf{R} = \begin{bmatrix} 1/3 & 0 & 0 \\ 1/3 & 0 & 0 \\ 1/9 & 1/9 & 1/3 \\ 0 & 1/3 & 0 \\ 0 & 1/3 & 0 \\ 0 & 1/3 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \mathbf{A} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

The computational and storage gains of using matrices $\mathbf{R}$ and $\mathbf{A}$ instead of $\mathbf{M}$ are very big:

Table 1: Storage Savings

Dataset	# of pages	M	R + A	H
cnr-2000	325557	42.57 GB	**8.209 MB**	51.56 MB
eu-2005	862664	588.8 GB	**34.42 MB**	300.1 MB
india-2004	1382908	131.6 GB	**38.82 MB**	268.7 MB
indochina	7414866	1.095 TB	**262.3 MB**	2.947 GB
uk-2002	18520486	1.323 TB	**604.5 MB**	4.580 GB

Storage Savings. Matrix $\mathbf{A}$ has only n non zero elements, all equal to 1. Thus, it is very easy to choose a compact storage mechanism that exploits its extreme sparseness. The number of non-zero elements of matrix $\mathbf{R}$ is by definition upper bounded by the number of non-zero elements of hyperlink matrix $\mathbf{H}$, but in realistic cases matrix $\mathbf{R}$ contains much less elements. In Table 1, we see the storing requirements[2] of these matrices for some of the datasets we experimented on (details about the datasets and the experimental methodology can be found in Section 4.1). Notice that after the factorisation, the combined storage needs for matrices $\mathbf{R}$ and $\mathbf{A}$ are significantly lower even when compared to the storage requirements of hyperlink matrix $\mathbf{H}$.

Computational Advantages. The factorization of matrix $\mathbf{M}$ also leads to considerable computational savings as well. If we assume the use of the Power Method for the extraction of the ranking vector, every step involves a calculation that looks like this:

$$\begin{aligned}\hat{\boldsymbol{\pi}}^{\intercal} &= \boldsymbol{\pi}^{\intercal}\mathbf{P} \\ &= \eta\boldsymbol{\pi}^{\intercal}\mathbf{H} + \mu\boldsymbol{\pi}^{\intercal}\mathbf{M} + (1-\eta-\mu)\boldsymbol{\pi}^{\intercal}\mathbf{E}\end{aligned}$$

The Matrix×Vector (MV) product $\boldsymbol{\pi}^{\intercal}\mathbf{M}$ needs to be computed in each iteration. Now, if we replace matrix $\mathbf{M}$ with the product $\mathbf{RA}$ we have a significant drop to the number of floating point operations needed. In particular:

Let $\Omega_{\mathbf{M}}$ denote the number of floating point operations involved in the direct computation of the product $\boldsymbol{\pi}^{\intercal}\mathbf{M}$ using matrix $\mathbf{M}$. If we make use of the factorization, we have the product $(\boldsymbol{\pi}^{\intercal}\mathbf{R})\mathbf{A}$. First, we compute the product $\boldsymbol{\pi}^{\intercal}\mathbf{R}$ and then, the resulting vector is multiplied with $\mathbf{A}$. If we denote $\Omega_{\mathbf{R}}$ and $\Omega_{\mathbf{A}}$ the corresponding number of operations, we get $\Omega_{\mathbf{R}\times\mathbf{A}} \triangleq \Omega_{\mathbf{R}} + \Omega_{\mathbf{A}}$ as the total number of operations for the second case.

Assuming that the number of operations of a Sparse Matrix Vector product (SpMV) is twice the number of non-zero elements of the sparse matrix involved and if we use $n_z(\cdot)$ to denote the number of non-zero elements of a matrix, we get:

$$\Omega_{\mathbf{M}} = 2n_z(\mathbf{M}) \text{ and } \Omega_{\mathbf{R}\times\mathbf{A}} = \Omega_{\mathbf{R}} + \Omega_{\mathbf{A}} \approx 2n_z(\mathbf{R})$$

Note that because of the special structure of matrix $\mathbf{A}$ and of the fact that it contains only zeros and ones, $\Omega_{\mathbf{A}}$ is computationally negligible, because it can be implemented in a way that it costs only n assignment operations. Thus, we see that

$$\Omega_{\mathbf{M}} = 2n_z(\mathbf{M}) \gg 2n_z(\mathbf{R}) \approx \Omega_{\mathbf{R}\times\mathbf{A}}$$

Actually, if we take into account the fact that matrices $\mathbf{A}$ and $\mathbf{R}$ are much smaller than $\mathbf{M}$ and more likely to fit into the main memory, we see that each iteration requires less disk I/Os, which by its own means, greatly accelerates the computation.

[2]The values of Table 1 assume the usage of *Harwell-Boeing* sparse matrix storage scheme.

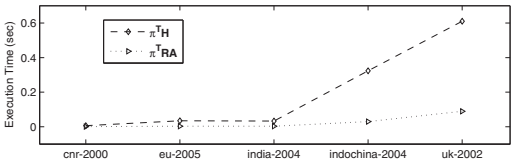


Figure 2: Execution time of the SpMV products, $\boldsymbol{\pi}^{\intercal}\mathbf{H}$ and $(\boldsymbol{\pi}^{\intercal}\mathbf{R})\mathbf{A}$ for a number of different datasets

Furthermore, observe that

$$\Omega_{\mathbf{H}} \triangleq 2n_z(\mathbf{H}) \gg 2n_z(\mathbf{R}) \approx \Omega_{\mathbf{R}\times\mathbf{A}}$$

which means that the bottleneck of each iteration is by far the computation of the SpMV product $\boldsymbol{\pi}^{\intercal}\mathbf{H}$ (involved to the computation of PageRank as well). Figure 2 provides experimental evidence of this fact. In this simple experiment we compare the execution times per iteration of these products for the five different datasets of Table 1 and we clearly see the significant time difference.

Finally, the product $(\boldsymbol{\pi}^{\intercal}\mathbf{R})\mathbf{A}$ of each iteration can be computed in a completely parallel or distributed fashion, with the introduction of very small communication cost. So, in the parallel case, the additional burden becomes negligible.

All these observations suggest that the computational cost induced by the inter-level proximity matrix, is practically very small and, as we will see in the following section, can be compensated by the fact that the introduction of $\mathbf{M}$ results in slightly faster convergence.

3.2.2 The Algorithm

Having defined the components of matrix $\mathbf{P}$, we are now ready to state the NCDawareRank algorithm. Because of the sparseness of the involving components and in order to be able to make direct comparisons with the standard PageRank algorithm, we suggest using the Power Method for the computation of the steady state probability distribution. The algorithm is given below:

Algorithm 1 NCDawareRank $(\mathbf{H}, \mathbf{A}, \mathbf{R}, \boldsymbol{\pi}^{\mathbf{0}}, \epsilon)$

1. Let the initial approximation be $\boldsymbol{\pi}^0$. Set $k = 0$.
2. Compute

$$\begin{aligned}\boldsymbol{\pi}^{(k+1)\intercal} &= \boldsymbol{\pi}^{(k)\intercal}\mathbf{P} \\ &= \eta\boldsymbol{\pi}^{(k)\intercal}\mathbf{H} + \mu\boldsymbol{\pi}^{(k)\intercal}\mathbf{RA} + (1-\eta-\mu)\boldsymbol{\pi}^{(k)\intercal}\mathbf{E}\end{aligned}$$

3. Normalize $\boldsymbol{\pi}^{(k+1)\intercal}$ and compute

$$r = \|\boldsymbol{\pi}^{(k+1)\intercal} - \boldsymbol{\pi}^{(k)\intercal}\|_1$$

If $r < \epsilon$, quit with $\boldsymbol{\pi}^{(k+1)\intercal}$, otherwise set $k = k+1$ and go to step 2.

It is known that the convergence rate of the Power Method applied to a stochastic matrix depends on the magnitude of the subdominant eigenvalue, $|\lambda_2|$. Precisely, the asymptotic convergence rate is the rate at which $|\lambda_2|^k \to 0$. The following theorem bounds the subdominant eigenvalue of the NCDawareRank matrix $\mathbf{P}$.

THEOREM 1. *The subdominant eigenvalue of matrix* $\mathbf{P}$ *involved in the NCDawareRank, defined by (2), is upper bounded by* $\eta + \mu$.

PROOF. We define the matrix

$$\mathbf{Z} \triangleq \frac{1}{\eta+\mu}(\eta\mathbf{H}+\mu\mathbf{M})$$

Notice that since matrices $\mathbf{H}$ and $\mathbf{M}$ are row-stochastic, matrix $\mathbf{Z}$ is also row-stochastic:

$$\begin{aligned}\mathbf{Ze} &= \frac{1}{\eta+\mu}(\eta\mathbf{H}+\mu\mathbf{M})\mathbf{e} \\ &= \frac{1}{\eta+\mu}(\eta\mathbf{He}+\mu\mathbf{Me}) \\ &= \frac{1}{\eta+\mu}(\eta\mathbf{e}+\mu\mathbf{e}) = \mathbf{e}\end{aligned}$$

Now, notice that we can express matrix $\mathbf{P}$ in terms of $\mathbf{Z}$ as follows:

$$\mathbf{P} = \alpha'\mathbf{Z} + (1-\alpha')\mathbf{E}$$

where $\alpha' \triangleq \eta+\mu < 1$ and $\mathbf{E} = \mathbf{ev}^\intercal$, for some probability vector with non-zero elements $\mathbf{v}^\intercal$.

But from the celebrated Google lemma (for a proof see [22]) we know that if the spectrum of the stochastic matrix $\mathbf{S}$ is $\{1, \lambda_2, \ldots, \lambda_n\}$ then the spectrum of matrix $\mathbf{G} = \alpha\mathbf{S}+(1-\alpha)\mathbf{ev}^\intercal$, where $\mathbf{v}^\intercal$ a probability vector with positive elements, is $\{1, \alpha\lambda_2, \ldots, \alpha\lambda_n\}$.

Thus, the subdominant eigenvalue of $\mathbf{P}$ is $(\eta+\mu)\lambda_2$ where λ_2 denotes the second largest eigenvalue of the stochastic matrix $\mathbf{Z}$. However, for every eigenvalue of a stochastic matrix, $|\lambda| \leq 1$ holds (see [28]), and the bound follows. □

Table 2 shows the number of iterations till convergence and the running time of NCDawareRank against PageRank for the different datasets of Table 1. To isolate the effect of the introduction of parameters η, μ we used the same teleportation matrix for both algorithms and we took as criterion of convergence a difference in the L_1-norm, lower than 10^{-5}. From the values of Table 2 we see that the introduction of parameter μ results in reduction of the number of iterations.

To further test this effect, we run NCDawareRank introducing increasingly big values of μ while keeping the sum $\eta+\mu$ constant. The results are presented in Table 3. Even for values of μ as low as 0.005, we have a drop of the number of iterations till convergence, and this drop continues as μ increases and then stabilizes. This is true for all the datasets we experimented on. These results and the fact that we want to avoid ignoring the direct link structure, suggest that a good choice for parameter μ is a value near 0.10.

Remark 2. Of course many other methods could be used to extract the steady state distribution of $\mathbf{P}$. In fact, because of the similarity in the definitions between PageRank and NCDawareRank, almost every algorithm or approach proposed for PageRank could easily be applied to the calculation of NCDawareRank as well.

Table 3: Iterations till Convergence for Different Values of Parameter μ

$\mu =$	0.005	0.01	0.02	0.05	0.1	0.2	0.3
cnr-2000	47	45	43	41	**40**	40	41
eu-2005	42	41	40	39	**38**	40	41
india-2004	47	46	45	42	**42**	42	42
indochina-2004	46	45	44	42	**42**	42	42
uk-2002	45	44	43	42	**41**	41	41

3.3 Block-wise Ranking

The factorisation of matrix $\mathbf{M}$ has another very useful side-effect: it makes it very easy to calculate a block-level ranking. For example, when the blocks correspond to websites, we can easily compute site ranking as a by-product of NCDawareRank. Let's see how.

In order to compute the probability of being in a given block, we have to construct a $N \times N$ stochastic matrix whose ij^{th} element denotes the probability of a transition from block $\mathcal{A}_I$ to block $\mathcal{A}_J$ (see [31]). This can be done in two steps: First we replace each row of each block $\mathbf{P}_{IJ}$ of matrix $\mathbf{P}$ by the sum of its elements, $\mathbf{P}_{IJ}\mathbf{e}$, and then we reduce each column subvector $\mathbf{P}_{IJ}\mathbf{e}$ to a scalar which will represent the total probability of leaving any page of block $\mathcal{A}_I$ to enter any page of block $\mathcal{A}_J$. To determine the latter probability we need to sum the elements of $\mathbf{P}_{IJ}\mathbf{e}$, after each of these elements have been weighted by the conditional probability of being in a particular state of block $\mathcal{A}_I$ given that we are in block $\mathcal{A}_I$.

These weighting factors can be obtained easily by the steady state probability vector of matrix $\mathbf{P}$. Let's assume that we partition the steady state vector according to the NCD blocks, $\boldsymbol{\pi} = (\boldsymbol{\pi}_1, \boldsymbol{\pi}_2, \ldots, \boldsymbol{\pi}_N)$, where each subvector $\boldsymbol{\pi}_I$ is equal to :

$$\boldsymbol{\pi}_I = (\pi_{I_1}, \pi_{I_2}, \ldots, \pi_{I_{|\mathcal{A}_I|}})$$

where π_{I_k} denotes the element of the steady state vector $\boldsymbol{\pi}$ corresponding to the kth page of the Ith NCD block. Then, the ij^{th} element of the resulting matrix, which we denote $\mathbf{C}$, will be given by:

$$C_{ij} \triangleq \frac{\boldsymbol{\pi}_I^\intercal}{\boldsymbol{\pi}_I^\intercal\mathbf{e}}\mathbf{P}_{IJ}\mathbf{e} \tag{3}$$

Remark 3. The matrix defined in (3) is quite similar to the coupling matrix of the NCD literature [31], except the fact that here we use the previously calculated stationary

Table 2: Number of Iterations till Convergence and Running Time Comparisons

	cnr-2000 325557 nodes		eu-2005 862664 nodes		india-2004 1382908 nodes		indochina-2004 7414866 nodes		uk-2002 18520486 nodes	
Algorithm	iter	time(sec)	iter	time(sec)	iter	time(sec)	iter	time(sec)	iter	time(sec)
NCDawareRank ($\eta = 0.75, \mu = 0.10$)	40	0.4759	38	1.8807	42	2.5576	42	20.3740	41	42.3371
PageRank (damping factor = 0.85)	48	0.6041	42	1.9989	48	2.7251	47	22.0154	46	44.9578
NCDawareRank ($\eta = 0.85, \mu = 0.10$)	107	1.2698	111	5.5493	114	6.7577	115	55.9905	113	116.3179
PageRank (damping factor = 0.95)	141	1.7698	122	5.9013	141	8.2676	136	63.9854	133	130.4432

distribution of matrix $\mathbf{P}$ instead of the approximations obtained by the independent solution of each NCD block.

The steady state probability vector of matrix $\mathbf{C}$, i.e the block-level ranking vector, can be obtained directly from the steady state vector of matrix $\mathbf{P}$ and matrix $\mathbf{A}$, without ever needing to explicitly compute $\mathbf{C}$.

THEOREM 2. *The unique steady state probability vector $\boldsymbol{\vartheta}$ of matrix $\mathbf{C}$, is given by*

$$\boldsymbol{\vartheta} = \boldsymbol{\pi}^{\intercal}\mathbf{A}^{\intercal}$$

where $\boldsymbol{\pi}$ is the stationary distribution of $\mathbf{P}$, and $\mathbf{A}$ is the matrix defined during the factorization of the inter-level proximity matrix $\mathbf{M}$.

PROOF. See Appendix A □

Thus, one can get block-wise ranking as a by-product of the computation of NCDawareRank, paying just a SpMV product.

4. EXPERIMENTS

4.1 Experimental Methodology

Throughout the paper we experiment with several medium and large sized snapshots of the Web obtained from the collection [5][3]. In particular, we used the `cnr-2000`, `eu-2005`, `india-2004`, `indochina-2004` and `uk-2002` Web graphs. Tables 1 and 2 provide information about these graphs. The larger graphs have been used for the storage need tests as well as for the computational and convergence comparisons of Section 3.2, whereas the medium sized ones for the sparsity and link spamming experiments presented in the following sections. During dataset preparation, we sorted the URLs lexicographically, we extracted the lengths of NCD blocks (which in our experiments correspond to websites) and we created matrices $\mathbf{R}$ and $\mathbf{A}$.

We compare NCDawareRank (with $\eta = 0.85$ and $\mu = 0.1$) against several other state-of-the-art link analysis algorithms. In particular:

- HyperRank (Baeza-Yates et al.[2]) with $\beta = 3$
- LinearRank (Baeza-Yates et al.[2]) with $L = 10$,
- PageRank (Page et al.[25]) using the canonical value for the damping factor, $\alpha = 0.85$.
- RAPr (Constantine and Gleich [10]) with the random variable A, following the $Beta(1, 1, [0, 1])$ distribution (the default distribution used by the authors in their publicly available implementation).
- TotalRank (Boldi [4]).

[3]These dataset are available at `http://law.dsi.unimi.it/`

These were the parameters we used for our experiments, with a few specifically stated exceptions.

In our tests in Section 4.2 we make use of the Kendall's τ correlation coefficient [20]; This is an intuitive nonparametric correlation index that has been widely used lately within the Web community (for example [2, 4, 19]) for ranking comparisons. The value of τ is 1 for perfect match and -1 for reversed ordering. All the experiments were done in Matlab on a 64bit machine with 12GB RAM.

4.2 Newly Added Pages and Sparsity

To show the performance of NCDawareRank in dealing with the problems caused by the low density of the Web-graph, we conduct the following two experiments using the `cnr-2000` and the `india-2004` datasets.

Newly Added Pages.

In the first experiment, we simulate the phenomenon of newly added pages by extracting the 90% of the incoming links of a set of randomly chosen pages. The altered graph then represents an "earlier version" of the Web, where these pages were new, and thus had fewer incoming links.

First, we run all the algorithms on the complete graph. Then we randomly choose a number of n pages for several values of n and we randomly remove 90% of their incoming links. Afterwards, we rebuild matrices $\mathbf{A}$ and $\mathbf{R}$ using the new hyperlink matrix, we re-run the algorithms and we calculate how different is the ordering induced on the new graph from the original. The measure used for this purpose is Kendall's τ correlation coefficient.

We repeat the above experiment 10 times; each time for all the different values of newly added pages and we present the average results in Table 4. High value of this metric means that the new ordering is very close to the ordering on the original graph, where all the links were included. We see that NCDawareRank outperforms all other algorithms, and its advantage becomes bigger as the number of newly added pages becomes larger. Thus, even they lack sufficient number of incoming links, the newly added pages get more appropriate ranking.

This was expected and is consistent with the way NCDawareRank views the Web. In NCDawareRank the importance of a page is not exclusively determined by its incoming links as in the other ranking methods; its "neighbourhood" also matters, since the existence of a link confers a small amount of rank to the corresponding ncd block. So, because the importance of a page is correlated with the importance of the website which contains it, even with fewer incoming links, more appropriate ranking is achieved.

Sparseness.

In this experiment, following the same methodology as before, we simulate the sparseness of the link graph by randomly selecting to include 90% - 40% of the links on the

Table 4: Ranking Performance for the Newly Added Pages Bias Problem Using the cnr-2000 Graph.

# New Pages	8000	10000	12000	15000	20000	25000	30000
HyperRank	94.51 ± 0.22	93.26 ± 0.19	92.96 ± 0.21	90.37 ± 0.30	87.72 ± 0.28	84.87 ± 0.31	82.34 ± 0.30
LinearRank	93.80 ± 0.48	92.60 ± 0.24	91.23 ± 0.28	89.41 ± 0.47	86.56 ± 0.44	83.41 ± 0.53	80.69 ± 0.49
NCDawareRank	$\mathbf{96.81 \pm 1.06}$	$\mathbf{96.48 \pm 1.10}$	$\mathbf{96.64 \pm 0.42}$	$\mathbf{95.44 \pm 1.39}$	$\mathbf{94.77 \pm 0.72}$	$\mathbf{93.31 \pm 1.59}$	$\mathbf{91.49 \pm 1.42}$
PageRank	93.68 ± 0.59	92.46 ± 0.30	91.04 ± 0.37	89.19 ± 0.55	86.33 ± 0.53	83.05 ± 0.63	80.26 ± 0.57
RAPr	94.16 ± 0.37	92.96 ± 0.20	91.64 ± 0.23	89.87 ± 0.49	87.15 ± 0.41	84.08 ± 0.45	81.47 ± 0.41
TotalRank	94.15 ± 0.38	92.94 ± 0.21	91.62 ± 0.25	89.84 ± 0.51	87.12 ± 0.43	84.01 ± 0.48	81.37 ± 0.44

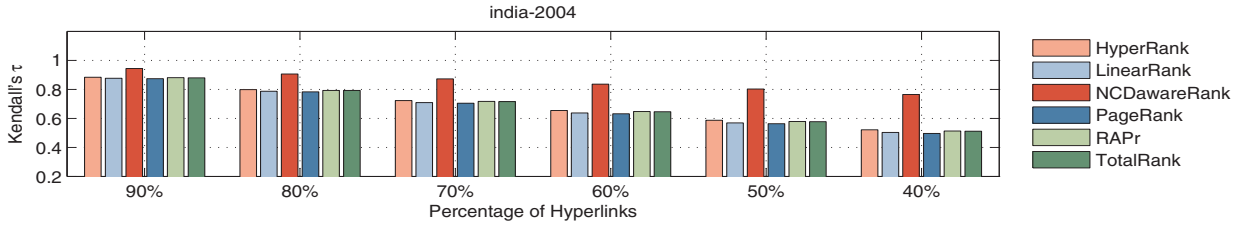


Figure 3: Testing the ranking quality on sparse versions of the india-2004 graph

altered graph and we compare the ranking results of the algorithms against their corresponding original rankings. In Figure 3 we see that NCDawareRank produces ranking that is more resistant to the effects of graph's sparseness when compared to the other ranking methods. Moreover, its advantage becomes bigger as the graph becomes sparser.

These results verify the intuition behind NCDawareRank; even though the direct link structure of the graph collapses with the exclusion of such many links, the inter level proximity captured by matrix $\mathbf{M}$ decays harder and thus preserves longer the coarser structure of the graph. This results in a ranking vector that proves less sensitive to the sparsity of the underlying space.

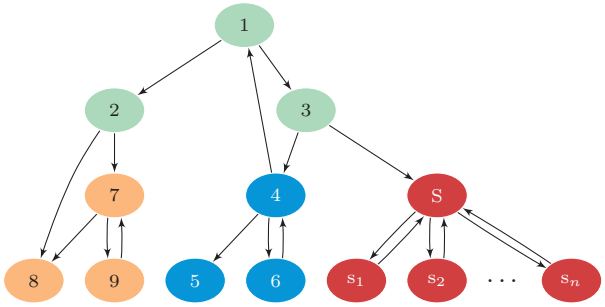


Figure 5: Example graph with spamming node S

4.3 Resistance to Manipulation

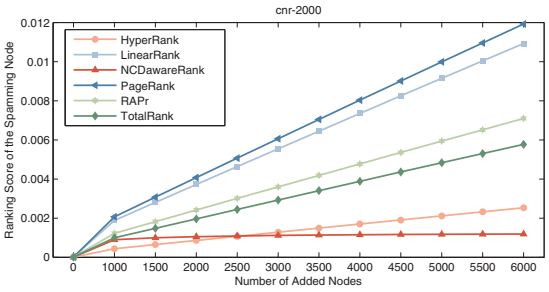
In this section we show how the intensity of manipulation affects NCDawareRank's ranking scores. The example graph of Figure 5 shows our approach to simulate the link spamming phenomenon. In this example, the spamming node S creates n nodes that funnel all their rank towards it. In fact, this is a standard technique commonly used in practice [13].

We follow the same approach in our experiments using the `cnr-2000` graph instead. In particular, we randomly pick a node with small initial ranking (from now on referred to as the "spamming node") and we add a number of n nodes that their only incoming link is from the spamming node and their only outgoing link is towards it, in the same manner as in the small example graph of Figure 5. Then, we run NCDawareRank and the other algorithms, for several values of n.

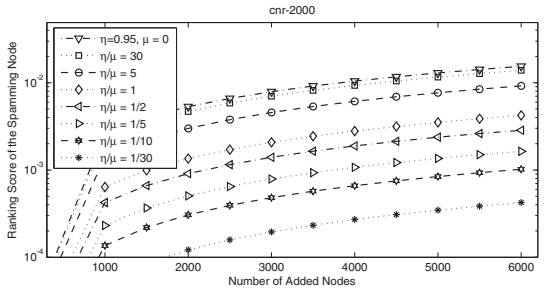
In Figure 4(a), we see the ranking score of the spamming node as a function of n, for all the different algorithms we experimented with. Notice that the anti-manipulation effect of NCDawareRank becomes immediately clear. The introduction of the matrix $\mathbf{M}$ and the ncd-based teleportation of our method, ensure that the ranking increment rate, after a small number of added nodes, becomes very small. So, in order to gain rank, a node has to have incoming links originated from other ncd blocks as well; the effect of nepotistic links is limited and thus, artificial boosting of the ranking score becomes harder.

In order to better isolate the resistance effect induced by the introduction of parameter μ and the corresponding matrix $\mathbf{M}$ alone, we run NCDawareRank using uniform teleportation, for several values of the ratio η/μ, while holding $\eta + \mu$ equal to 0.95, and we plot the results in Figure 4(b). We see that even with the introduction of a very small μ, NCDawareRank starts to exhibit positive resistance properties with the anti-manipulation effect increasing as the ratio η/μ tends to zero, as expected.

Of course as η/μ is getting more close to zero, the direct link structure of the Web values less, because the hyperlink matrix $\mathbf{H}$ gets increasingly ignored. However, the increased sensitivity of the spamming node's ranking score for small



(a) Ranking of the spamming node for different algorithms



(b) NCDawareRank's sensitivity to ratio η/μ

Figure 4: Ranking of the spamming node as a function of the added nodes n, for the cnr-2000 graph

values of the ratio η/μ, could lead to interesting alternative uses of our measure. For example, this characteristic could be exploited by a machine learning framework for better spam classification. This is an interesting research direction we are currently pursuing, with the results so far being very promising.

5. CONCLUSIONS

As already stated in the introduction, the near complete decomposability of the Web has primarily been regarded as a property that could be exploited to compute PageRank faster, or in a distributed manner.

Most approaches that exploit Web's hierarchical nature for ranking, typically involve the creation of a new, coarser level Web graph, on which afterwards, standard ranking technologies are applied. HostRank and DirRank algorithms proposed by Eiron et al.[13] follow this method. Xue et al.[32] construct a multi-layer hierarchical graph and introduce a random walk model on this layered graph, which they use to calculate the importance of the coarser layer first and then distribute this importance to the page layer.

NCDawareRank differs significantly, because it builds directly on the PageRank model using the complete Web-graph, instead of aggregated versions that may hide interesting aspects of its direct link structure. Furthermore, unlike other hierarchical techniques, our framework has the flexibility to bring together heterogeneous decompositions of the underlying space on the same model. For example, one can incorporate different criteria of decomposition like the language of a Web document, the category of its content, or perhaps coarser level structural decompositions by simply introducing new interlevel proximity matrices $\mathbf{M}_1, \mathbf{M}_2, \ldots$ and associated parameters. This opens new and interesting research paths that remain to be explored.

In this paper we considered the single level decomposition case. Our experiments suggest that the introduction of the inter-level proximity matrix involved in NCDawareRank can help alleviating known problems many state-of-the-art link analysis algorithms share, including biased ranking of newly added pages, and susceptibility to direct manipulation through link spamming. Our method introduces a new approach to the Web ranking problem and presents what we believe to be a mathematically attractive and elegant generalization of the basic PageRank model; a generalization that provides an intuitively appealing and computationally efficient theoretical framework for the exploitation of Web's decomposable nature.

6. REFERENCES

[1] K. Avrachenkov, N. Litvak, and K. S. Pham. Distribution of pagerank mass among principle components of the web. In *Proceedings of the 5th international conference on Algorithms and models for the web-graph*, WAW'07, pages 16–28, Berlin, Heidelberg, 2007. Springer-Verlag.

[2] R. Baeza-Yates, P. Boldi, and C. Castillo. Generalizing pagerank: damping functions for link-based ranking algorithms. In *Proceedings of the 29th annual international ACM SIGIR conference on Research and development in information retrieval*, SIGIR '06, pages 308–315, New York, NY, USA, 2006. ACM.

[3] K. Bharat, B.-W. Chang, M. R. Henzinger, and M. Ruhl. Who links to whom: Mining linkage between web sites. In *Proceedings of the 2001 IEEE International Conference on Data Mining*, ICDM '01, pages 51–58, Washington, DC, USA, 2001. IEEE Computer Society.

[4] P. Boldi. Totalrank: ranking without damping. In *Special interest tracks and posters of the 14th international conference on World Wide Web*, WWW '05, pages 898–899, New York, NY, USA, 2005. ACM.

[5] P. Boldi, B. Codenotti, M. Santini, and S. Vigna. Ubicrawler: A scalable fully distributed web crawler. *Software: Practice & Experience*, 34(8):711–726, 2004.

[6] P. Boldi, M. Santini, and S. Vigna. A deeper investigation of pagerank as a function of the damping factor. In *Web Information Retrieval and Linear Algebra Algorithms*, 2007.

[7] P. Boldi, M. Santini, and S. Vigna. Pagerank: Functional dependencies. *ACM Trans. Inf. Syst.*, 27(4):19:1–19:23, Nov. 2009.

[8] A. Cevahir, C. Aykanat, A. Turk, and B. Cambazoglu. Site-based partitioning and repartitioning techniques for parallel pagerank computation. *Parallel and Distributed Systems, IEEE Transactions on*, 22(5):786–802, 2011.

[9] J. Cho and S. Roy. Impact of search engines on page popularity. In *Proceedings of the 13th international conference on World Wide Web*, WWW '04, pages 20–29, New York, NY, USA, 2004. ACM.

[10] P. Constantine and D. Gleich. Random alpha pagerank. *Internet Mathematics*, 6(2):189–236, 2009.

[11] P.-J. Courtois. *Decomposability: Queueing and Computer System Applications*. ACM monograph series. Academic Press, 1977.

[12] N. Eiron and K. S. McCurley. Locality, hierarchy, and bidirectionality in the web. In *In Workshop on Algorithms and Models for the Web Graph*, 2003.

[13] N. Eiron, K. S. McCurley, and J. A. Tomlin. Ranking the web frontier. In *Proceedings of the 13th international conference on World Wide Web*, WWW '04, pages 309–318, New York, NY, USA, 2004. ACM.

[14] M. Faloutsos, P. Faloutsos, and C. Faloutsos. On power-law relationships of the internet topology. *SIGCOMM Comput. Commun. Rev.*, 29(4):251–262, Aug. 1999.

[15] D. F. Gleich, P. G. Constantine, A. D. Flaxman, and A. Gunawardana. Tracking the random surfer: empirically measured teleportation parameters in pagerank. In *Proceedings of the 19th international conference on World wide web*, WWW '10, pages 381–390, New York, NY, USA, 2010. ACM.

[16] M. R. Henzinger, R. Motwani, and C. Silverstein. Challenges in web search engines. In *Proceedings of the 18th international joint conference on Artificial intelligence*, IJCAI'03, pages 1573–1579, San Francisco, CA, USA, 2003. Morgan Kaufmann Publishers Inc.

[17] I. C. F. Ipsen and S. Kirkland. Convergence analysis of a pagerank updating algorithm by Langville and Meyer. *SIAM J. Matrix Anal. Appl.*, 27(4):952–967, Dec. 2005.

[18] S. Kamvar and T. Haveliwala. The condition number of the pagerank problem. *Stanford University Technical Report*, 2003.

[19] S. D. Kamvar, T. H. Haveliwala, C. D. Manning, and G. H. Golub. Exploiting the block structure of the web for computing pagerank. In *Stanford University Technical Report*, 2003.

[20] M. Kendall and J. Gibbons. *Rank Correlation Methods*. Oxford University Press, USA, 1990.

[21] A. N. Langville and C. D. Meyer. Updating pagerank with iterative aggregation. In *Proceedings of the 13th international World Wide Web conference on Alternate track papers & posters*, WWW Alt. '04, pages 392–393, New York, NY, USA, 2004. ACM.

[22] A. N. Langville and C. D. Meyer. *Google's PageRank and Beyond: The Science of Search Engine Rankings*. Princeton University Press, Princeton, NJ, 2006.

[23] C. D. Meyer. Stochastic complementation, uncoupling markov chains, and the theory of nearly reducible systems. *SIAM Rev.*, 31(2):240–272, June 1989.

[24] A. Ntoulas, M. Najork, M. Manasse, and D. Fetterly. Detecting spam web pages through content analysis. In *Proceedings of the 15th international conference on World Wide Web*, WWW '06, pages 83–92, New York, NY, USA, 2006. ACM.

[25] L. Page, S. Brin, R. Motwani, and T. Winograd. The pagerank citation ranking: Bringing order to the web. Technical Report 1999-66, Stanford InfoLab, November 1999.

[26] G. Pandurangan, P. Raghavan, and E. Upfal. Using pagerank to characterize web structure. In O. Ibarra and L. Zhang, editors, *Computing and Combinatorics*, volume 2387 of *Lecture Notes in Computer Science*, pages 1–4. Springer Berlin / Heidelberg, 2002.

[27] I. Pultarová and I. Marek. Convergence of multi-level iterative aggregation-disaggregation methods. *J. Computational Applied Mathematics*, 236(3):354–363, 2011.

[28] E. Seneta. *Non-negative Matrices and Markov Chains*. Springer Verlag, 2006.

[29] H. Simon and A. Ando. Aggregation of variables in dynamic systems. *Econometrica: Journal of The Econometric Society*, pages 111–138, 1961.

[30] H. A. Simon. *The Sciences of the Artificial (3rd ed.)*. MIT Press, Cambridge, MA, USA, 1996.

[31] W. J. Stewart. *Probability, Markov Chains, Queues, and Simulation: The Mathematical Basis of Performance Modeling*. Princeton University Press, Princeton, NJ, USA, 2009.

[32] G.-R. Xue, Q. Yang, H.-J. Zeng, Y. Yu, and Z. Chen. Exploiting the hierarchical structure for link analysis. In *Proceedings of the 28th annual international ACM SIGIR conference on Research and development in information retrieval*, SIGIR '05, pages 186–193, New York, NY, USA, 2005. ACM.

[33] Y. Zhu, S. Ye, and X. Li. Distributed pagerank computation based on iterative aggregation-disaggregation methods. In *Proceedings of the 14th ACM international conference on Information and Knowledge Management*, CIKM '05, pages 578–585, New York, NY, USA, 2005. ACM.

APPENDIX

A. PROOF OF THEOREM 2

THEOREM 2. *The unique steady state probability vector $\boldsymbol{\vartheta}$ of matrix $\mathbf{C}$, is given by*

$$\boldsymbol{\vartheta} = \boldsymbol{\pi}^\intercal \mathbf{A}^\intercal$$

where $\boldsymbol{\pi}$ is the stationary distribution of $\mathbf{P}$, and $\mathbf{A}$ is the matrix defined during the factorization of the inter-level proximity matrix $\mathbf{M}$.

PROOF. It can be proven that the matrix defined by (3) is a stochastic irreducible matrix [23] and hence it possesses a unique stationary equilibrium distribution. We will show that this probability distribution is $\boldsymbol{\vartheta}$, defined above.

It suffices to verify that equations $\boldsymbol{\vartheta}\mathbf{C} = \boldsymbol{\vartheta}$ and $\boldsymbol{\vartheta}\mathbf{e} = 1$ hold. This can be seen by straightforward calculations:

$$\begin{aligned}
\boldsymbol{\vartheta}\mathbf{C} &= \boldsymbol{\pi}^\intercal \mathbf{A}^\intercal \mathbf{C} \\
&= \boldsymbol{\pi}^\intercal \mathbf{A}^\intercal \begin{bmatrix} \frac{\boldsymbol{\pi}_1^\intercal}{\boldsymbol{\pi}_1^\intercal \mathbf{e}} & \cdots & \mathbf{0} \\ \vdots & \ddots & \vdots \\ \mathbf{0} & \cdots & \frac{\boldsymbol{\pi}_N^\intercal}{\boldsymbol{\pi}_N^\intercal \mathbf{e}} \end{bmatrix} \mathbf{P} \begin{bmatrix} \mathbf{e} & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & \mathbf{e} & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & \mathbf{e} \end{bmatrix} \\
&= \boldsymbol{\pi}^\intercal \mathbf{P} \mathbf{A}^\intercal = \boldsymbol{\pi}^\intercal \mathbf{A}^\intercal = \boldsymbol{\vartheta}
\end{aligned}$$

and

$$\boldsymbol{\vartheta}\mathbf{e} = \boldsymbol{\pi}^\intercal \mathbf{A}^\intercal \mathbf{e} = \boldsymbol{\pi}^\intercal \mathbf{e} = 1$$

Thus, probability vector $\boldsymbol{\vartheta}$ is the unique steady state solution of matrix $\mathbf{C}$, and the proof is complete. □